Transaction Cost-Aware Deep Reinforcement Learning for Financial Portfolio Optimization

Habtamu Tadese Sefera

A thesis proposal submitted to the Department of Computer Science and Engineering

School of Electrical Engineering and Computing

Office of Graduate Studies

Adama Science and Technology University

January, 2026

Adama, Ethiopia

Transaction Cost-Aware Deep Reinforcement Learning for Financial Portfolio Optimization

Habtamu Tadese Sefera

Advisor: Dr.-Ing. Frezewd Lemma

A thesis proposal submitted to the Department of Computer Science and Engineering,

School of Electrical Engineering and Computing

Office of Graduate Studies

Adama Science and Technology University

January, 2026

Adama, Ethiopia

Declaration

I declare that this thesis proposal, entitled "Transaction Cost-Aware Deep Reinforcement Learning for Financial Portfolio Optimization" is my own work and has not been submitted to any university for a similar purpose. The references used in this proposal are duly recognized by proper citations.

_____	_____	_____
Name of Student	Signature	Date

Recommendation of Advisors

I, the major advisor of this research proposal, hereby certify that I have closely advised the student while developing this proposal and read the draft thesis proposal entitled "Transaction Cost-Aware Deep Reinforcement Learning for Financial Portfolio Optimization" prepared under our guidance by Habtamu Tadese Sefera. Therefore, I recommend the submission of the proposal to the department for further review and evaluation.

_____	_____	_____
Major Advisor	Signature	Date

Approval Page

We hereby certify that the recommendation and suggestion given by the proposal review committee are appropriately incorporated into the final thesis proposal entitled "Transaction Cost-Aware Deep Reinforcement Learning for Financial Portfolio Optimization" prepared under our guidance by Habtamu Tadese Sefera. Therefore, we recommend the submission of proposal to the department for further review and evaluation.

_____	_____	_____
Major Advisor	Signature	Date

Approval of Board of Reviewers

We, the undersigned, members of the Board of Reviewers of the proposal open defense by Habtamu Tadese Sefera have read and evaluated the thesis proposal entitled "Transaction Cost-Aware Deep Reinforcement Learning for Financial Portfolio Optimization" and assessed the understanding of the candidate about the proposed research. This is, therefore, to certify that the thesis proposal is accepted and we recommend the implementation of the proposal.

Chairperson	Signature	Date
Reviewer 1	Signature	Date
Reviewer 2	Signature	Date

Final approval and acceptance of the thesis proposal is contingent upon submission of its final copy to the Office of Postgraduate Studies (OPGS) through the Department Graduate Council (DGC) and School Graduate Committee (SGC).

Department Head	Signature	Date
School Dean	Signature	Date
Office of Postgraduate Studies, Dean	Signature	Date

Contents

List of Figures ix
List of Tables x
List of Acronyms xi
Summary xii
CHAPTER ONE 1
1. INTRODUCTION 1
1.1 Background of The Study 1
1.2 Motivation 1
1.3 Statement of The Problem 2
1.4 Research Questions 2
1.5 Objective of The Study 3
1.5.1 General Objective 3
1.5.2 Specific Objectives 3
1.6 Significance of The Study 3
1.7 Scope of The Study 3
1.8 Limitations 3
CHAPTER TWO 4
2. LITERATURE REVIEW 4
2.1 Evolution of Portfolio Theory 4
2.2 Deep Reinforcement Learning in Finance 4
2.3 Transaction Cost-Awareness and Risk Integration in DRL 5
CHAPTER THREE 6
3. DESIGN AND METHODOLOGY 6
3.1 Problem Formulation 6
3.1.1 State Space ($\mathcal{S}$) 6
3.1.2 Action Space ($\mathcal{A}$) 6
3.1.3 Transaction Dynamics 6
3.1.4 Transaction Cost Model 7
3.2 Cost-Aware Reward Function 7
3.3 Algorithm Architecture: DDPG / PPO 8

3.3.1 Actor Network 8
3.3.2 Critic Network 8
3.3.3 Cost-Aware Integration 8
3.4 Data and Preprocessing 9
3.5 Evaluation Metrics 10
3.6 Development Tools 10
3.6.1 Hardware Specifications 10
3.6.2 Software Specification 11
3.7 Expected Outcome 11
CHAPTER FOUR 12
4. WORK AND BUDGET PLAN 12
4.1 Time Schedule 12
4.2 Budget Breakdown 13
Reference 14

List of Figures

Figure 1. Proposed System Model 8
Figure 2. Algorithm Architecture 9

List of Tables

Table 4.1. Time plan of research work 12
Table 4.2. Budget breakdown for research work 13

List of Acronyms

ARR	Accumulated Rate of Return
DDPG	Deep Deterministic Policy Gradient
DRL	Deep Reinforcement Learning
MDP	Markov Decision Process
MDD	Maximum Drawdown
MPT	Modern Portfolio Theory
MVO	Mean-Variance Optimization
OHLCV	Open, High, Low, Close, Volume
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
ROI	Return on Investment
RSI	Relative Strength Index
SAC	Soft Actor-Critic

Summary

This research introduces a Transaction Cost-Aware Deep Reinforcement Learning (DRL) framework for optimizing financial portfolios as a way of overcoming the drawbacks of the traditional Modern Portfolio Theory (MPT) and standard DRL models. Although MPT, a concept originally developed by Markowitz in 1952, is still the basis for mean-variance under idealized situations, the real-world markets show non-linear behaviors, volatility clustering, and a great deal of friction from transactions that can easily ruin the performance of unconstrained DRL agents who are, thus, prone to excessive trading ("churning") and, eventually, net underperformance. The proposed framework approaches portfolio management as a Markov Decision Process and incorporates proportional transaction costs into a modified logarithmic reward function, enabling the agent to develop disciplined and adaptive rebalancing strategies that not only maximize return but also minimize turnover in a data-driven way, therefore, replicating the theoretical "no-trade region" approximation. The implementation consists of employing actor-critic algorithms (PPO/DDPG) in the FinRL environment with historical daily data ranging from 2015 to 2023, either from stocks or cryptocurrencies. This framework will be put to the test against baselines (Buy-and-Hold, Mean-Variance) using metrics such as Sharpe Ratio, Maximum Drawdown, and Turnover Rate with the intention of proving not only the superiority of its risk-adjusted net returns but also the practical viability of this approach for automated trading systems.

Keywords: Portfolio Optimization, DRL, MDP, Transaction cost, Cost-Aware Reward Function, Actor-Critic Algorithms, PPO, DDPG, Sharpe Ratio, Maximum Drawdown, Turnover Rate

CHAPTER ONE

1. INTRODUCTION

1.1 Background of The Study

The main goal of financial portfolio management is to wisely spread the money among different types of assets---such as equities, fixed-interest securities, safe, and high-risk investments like digital currencies to get the best possible return and at the same time to control the risk exposure. Modern Portfolio Theory (MPT), developed by Harry Markowitz in his canonical paper on portfolio choice, has been the foundation of this area for more than 70 years(Markowitz, 1952). MPT brought forward the Mean-Variance Optimization method, which mathematically characterizes the "efficient frontier"---a group of portfolios, which, for a certain amount of risk, yield the highest expected return. This method greatly changed the way of practicing investing as it measured diversification gains and laid down a solid base for asset allocation.

Modern Portfolio Theory (MPT) has prevailed until now, and still, MPT is based on a number of unrealistic assumptions, such as market efficiency, normally distributed returns, linear correlation of assets, and rationality of investors. However, in reality, there are big differences between the financial markets and these ideal situations, such as volatility clustering, fat-tailed return distributions, non-stationary dynamics, and behavioral biases among participants. These complexities of the real world often make traditional mean-variance solutions either suboptimal or fragile during market stress.

To solve the problems mentioned above, Reinforcement Learning (RL) and especially Deep Reinforcement Learning (DRL) have become well-known as an effective and powerful alternative paradigm. RL abstracts portfolio management as a series of decisions made over time in a Markov Decision Process, where at each point the agent sees a certain state of the market (e.g., price histories and economic indicators), takes certain actions (portfolio rebalancing), and gets some rewards according to the risk-adjusted performance. The agent's learning of adaptive strategies characterized by non-linearity and sharp turns without being dependent on limited parametric assumptions is made possible by the use of this dynamic, model-free methodology(Z. Jiang et al., 2017). The traditional methods can hardly cope with the volatile environment of cryptocurrency markets, while empirical studies have already shown DRL's excellence in such markets.

1.2 Motivation

Deep Reinforcement Learning (DRL) has been successful in simulated environments, but there is still one major challenge that hinders its application in real life: the transaction costs are either overlooked or underestimated(Coriat & Benhamou, 2025). In the case of frictionless simulations, the DRL agents can take the small price inefficiencies through high-frequency rebalancing, and thus, they are earning more returns than realistic. However, the real trading is subjected to

unavoidable frictions, which are brokerage fees, bid-ask spreads, slippage, taxes, and market impact---which consequently cut profits proportional to the level of trading activity.

The learning of very aggressive policies is typical for the class of Unconstrained DRL models, which is referred to as "churning" and is characterized by excessive turnover. This over-trading creates a gross alpha but usually ends up being net underperformance after costs, thus making the strategy not feasible for live trading(Liang et al., 2018). This research is driven by the necessity of explicit use of the realistic market frictions in the DRL framework. The agent can be led through the use of various cost-aware mechanisms and a reward function designed to penalize turnover, to learn disciplined and cost-efficient policies that trade only when the expected returns greatly outnumber the execution costs(Y. Jiang et al., 2024).

1.3 Statement of The Problem

Financial portfolio optimization is a process of making decisions that are not only sequential but also continuous and heavily dependent on the inherent market uncertainty. Although Deep Reinforcement Learning (DRL) models have been able to demonstrate their capability of learning adaptive trading policies, they still come across a major problem: Traditional DRL portfolio optimization methods often optimize gross logarithmic returns or Sharpe ratios in hypothetical environments, which do not take into account or do not sufficiently penalize the real-world trading frictions, particularly transaction costs. This oversight results in strategies that look optimal in back tests but underperform when implemented, as profits are chipped away by the constant rebalancing. This paper focuses instead on the fundamental problem of algorithmic over-trading, that is, excessive turnover without regard to costs, which dilutes realizable returns. This research addresses this gap by introducing a transaction cost-aware DRL framework which incorporates the trade-off between rebalancing benefits (e.g., risk aversion and opportunity pursuit) and execution costs to enable robust and deployable policies.

1.4 Research Questions

RQ1: How does adding transaction costs to the reward function of a Deep Reinforcement Learning (DRL) model affect portfolio turnover and net returns compared to a DRL model that ignores transaction costs?

RQ2: Can a transaction cost-aware DRL model achieve better risk-adjusted performance (e.g., Sharpe Ratio and Maximum Drawdown) than traditional portfolio strategies such as Buy-and-Hold and Mean-Variance Optimization?

RQ3: Does including transaction costs in the DRL reward function lead to less frequent portfolio rebalancing, making the learned trading strategy more practical for real-world use?

1.5 Objective of The Study

1.5.1 General Objective

To develop, implement, and assess a Transaction Cost-Aware Deep Reinforcement Learning framework for effective financial portfolio management in realistic market conditions.

1.5.2 Specific Objectives

- To quantify the detrimental effects of transaction cost on the performance of conventional DRL agent.
- To design a modified reward function that directly penalize excessive trading while preserving return maximization.
- To implement actor-critic DRL algorithms (e.g., Deep Deterministic Policy Gradient (DDPG) or Proximal Policy Optimization (PPO) for continuous portfolio allocation under explicit cost constraint.

1.6 Significance of The Study

This research holds multifaceted importance:

- Academic Contribution: It adds to the growing literature on DRL in quantitative finance and addresses market frictions in the learning model, to narrow the disconnect between theoretical models and real-world applicability (Z. Jiang et al., 2017; Liang et al., 2018).
- Practical implication for Algorithmic Trading: The proposed strategy can be used as a basis for real-world implementations of resilient automated systems with an emphasis on avoiding overfitting to noise in favor of upholding profitable trades.
- Benefits for Investors and Practitioners: By filtering out unprofitable trading noise, the method strengthens more stable risk adjusted returns, critical for retail investors and fund managers in volatile or high-cost markets.

1.7 Scope of The Study

The research is focused on applying one of the DRL algorithms, DDPG or PPO, to meet the cost-aware objectives. DDPG or PPO. Portfolios are built from daily historical data on the basis of some 'typical' assets, for example, constituents of leading indices (e.g., Dow Jones Industrial Average tech stocks) or highly volatile assets (e.g., cryptocurrencies), across a period of around roughly 8 years of market history.

1.8 Limitations

Assumptions include sufficient market liquidity for instantaneous execution at observed prices and a proportional transaction cost model (fixed percentage of turnover). The framework does not explicitly model extreme events (e.g., flash crashes, liquidity droughts, or regulatory changes) where slippage could be amplified or trades impossible. Results are based on historical back testing and may not fully capture forward-looking risks.

CHAPTER TWO

2. LITERATURE REVIEW

2.1 Evolution of Portfolio Theory

The modern premise of portfolio theory comes from Markowitz, who in 1952 succeeded in providing the Mean-Variance Optimization (MVO) paradigm, which eventually led to the development of the efficient frontier, portraying the best trade-off between the expected return and risk through diversification (Markowitz, 1952). This research not only founded but also widely spread the principle of diversification, resulting in such developments as the Capital Asset Pricing Model (CAPM) and multi-factor models.

Nevertheless, the frictionless approach taken by MPT made its implementation difficult. The earlier extensions that accounted for transaction costs demonstrated that a cost based on a percentage formed a "no-trade region" where investors were willing to put up with portfolio drift in order to avoid costly rebalancing unless profits surpassed (Constantinides, 1986)(Davis & Norman, 1990). Davis and Norman (1990) offered rigorous continuous-time solutions, determining explicit frontiers for this area and proving that the best policies include rare interventions.

More recent advances dealt with multi-periods having fixed/proportional costs, liquidity constraints, and stochastic control, which often made numerical methods necessary due to the complexity of the problem. These findings pointed to the necessity for adaptable, approximate solutions, thus allowing the introduction of machine learning methods that learn from data without completely solving.

2.2 Deep Reinforcement Learning in Finance

The first instances of Reinforcement Learning (RL) in finance were using only tabular methods; however, Deep Reinforcement Learning (DRL) made it possible to conduct processing of high-dimensional market data and the continuous actions necessary for the utmost precision in weight allocation. Jiang et al. (2017) were the ones who developed a complete DRL framework with ensemble critics and direct policy learning from price sequences; then, the results in equities and cryptocurrencies were even better (Z. Jiang et al., 2017). This led to a variety of architectures: PPO was chosen for its stability, SAC was for its exploration through the use of entropy regularization, and recurrent/attention models were for the detection of both temporal and cross-asset dependencies. The FinRL library made standard environments that had to deal with real-life constraints such as transaction costs and risk aversion, thus making it possible to perform reproducible benchmarking across different markets (Liu et al., 2022).

Newer studies highlight exploration and robustness as their main focus. Multimodal DRL fuses news sentiment, macroeconomic data, and alternative signals for richer states (Nawathe et al.,

2024). Attention mechanism-based methods are able to capture the changing correlations (Xue & Ye, 2025). Among the industry-focused frameworks, one that deals with successfully deploying AI in production mentioned scalability and the sim-to-real gaps as issues (Ndikum & Ndikum, 2024). Also, the regime-aware models are capable of identifying market shifts and thereby devising adaptive strategies. The comprehensive surveys that have been conducted observe the superiority of DRL in non-stationary environments but at the same time they point out the need for generalization and interpretability (Bai et al., 2024).

2.3 Transaction Cost-Awareness and Risk Integration in DRL

The first DRL models neglected to account for frictions which caused them to produce unrealistic, high-turnover policies. The work by Liang et al. (2018) led to the strengthening of the model through adversarial training, which worked indirectly to reduce churning (Liang et al., 2018).

Through the imposition of transaction costs as reward shaping---subtracting proportional penalties from returns has become widespread, thus inducing agents to simulate the approximated regions of the theoretical no-trade based on data. The environment of FinRL natively allows such penalties, thus resulting in the implementation of disciplined trading (Liu et al., 2022).

Recent studies refine these mechanisms:

- Attention-enhanced Dirichlet policies with penalties for cost and variance result in realistic turnover and higher risk-adjusted returns (Xue & Ye, 2025).
- Risk-aware frameworks apply drawdown control and slippage to enable active optimization (Lwele et al., 2025).
- Regime-adaptive RL merges detection with cost-obligated rewards during the volatile times.
- Industry-grade approaches embed fees, market impact, and constraints for practical deployment (Ndikum & Ndikum, 2024).
- Multimodal integrations maintain cost penalties across diverse inputs (Nawathe et al., 2024).

Surveys confirm that cost-aware methods dramatically reduce turnover while keeping or even improving Sharpe ratios and drawdown metrics over frictionless baselines and traditional methods(Bai et al., 2024).

Overall, the literature reflects a maturation: classical no-trade principles merge with DRL's adaptability, with 2024--2025 advancements focusing on risk sensitivity, multimodal data, and deployability to bridge simulation-to-reality gaps.

CHAPTER THREE

3. DESIGN AND METHODOLOGY

In this section, we present the mathematical backbone as well as the algorithmic structure required for solving the portfolio optimization problem. By treating the financial market as a Markov Decision Process (MDP), the Deep Reinforcement Learning (DRL) agent is allowed to gradually discover the best strategy for long-term portfolio growth that not only is maximized but also accepts the very realistic frictions like transaction costs.

3.1 Problem Formulation

The portfolio management process is represented as a finite-horizon MDP over discrete time intervals $(t = 0, 1, \dots, T)$, whereby the agent at each time-step rebalances the portfolio in accordance with the prevailing market conditions that are being observed.

3.1.1 State Space ($\mathcal{S}$)

The state space (s_t) includes the important market data and the existing portfolio movement to guide the rebalancing decisions. It consists of historical price data from different features (e.g., Open, High, Low, Close, and Volume) for (M) risky assets over a look-back period of length (k), together with the earlier portfolio weights.

$$s_t = [X_{t-k:t}, w_{t-1}]$$

In this equation, $(X_{t-k:t} \in R^{M \times k \times f})$ is a (f)-dimensional tensor containing price/features per asset and usually $(f \geq 4)$, and $(w_{t-1} \in R^{M+\mathbb{1}})$ is the portfolio weight vector for the previous period, including a cash holding $(w_{t-1,0})$.

This formulation adheres to established practices in DRL for finance (Z. Jiang et al., 2017).

3.1.2 Action Space ($\mathcal{A}$)

The action (a_t) defines the target portfolio weights after rebalancing, representing the desired allocation of total capital.

$$a_t = w_t = \left[w_{t,0}, w_{t,1}, \dots, w_{t,M}\right]^T$$

With constraints: $(\sum_{i=0}^{M} w_{t,i} = 1)$and $(w_{t,i} \geq 0)$ no short-selling; $(w_{t,0})$is cash. The continuous action space is handled by normalizing raw network outputs (e.g., via softmax) to enforce the simplex constraint.

3.1.3 Transaction Dynamics

Between rebalancing periods, asset prices evolve according to market returns$(y_t \in R^{M+\mathbb{1}})$ with $(y_{t,0} = 1)$ for cash.

The pre-rebalancing weights adjust passively to $w'_t = \frac{w_{t-1} \odot y_t}{\sum(w_{t-1} \odot y_t)}$, where $\odot$ denotes element-wise multiplication. After applying (w_t) the portfolio value updates as described in the reward section.

3.1.4 Transaction Cost Model

Trading incurs proportional costs, modeled as a fixed rate λ (e.g., 0.1% or 0.001) applied to turnover.

A precise formulation accounts for price movements between periods:

Where V_t^+ is the portfolio value just before rebalancing (after price changes). However, a common approximation simplifies to:

$$Cost_t = \lambda \sum_{i=0}^{M} \left| w_{t,i} \cdot V_t^+ - w'_{t,i} \cdot V_t^+ \right|$$

Ignoring cash turnover and assuming small costs relative to returns. This penalizes changes in risky asset allocations post-price adjustment (Jiang et al., 2017; Liang et al., 2018).

3.2 Cost-Aware Reward Function

The key innovation lies in a reward that explicitly discourages excessive trading ("churning") by deducting transaction costs from gross returns, encouraging the agent to balance return-seeking with turnover minimization.

Standard DRL agents often use logarithmic returns ($r_t = \ln(V_t/V_{t-1})$, which promote growth via the Kelly criterion. The proposed cost-aware variant integrates costs directly into portfolio evolution (Liang et al., 2018):

$$V_t = V_{t-1} \cdot (w_{t-1}^T y_t) \cdot \left(1 - \lambda \sum_{i=1}^{M} \left| w_{t,i} - w'_{t-1,i} \right| \right)$$

The instantaneous reward is then:

$$R_t = \ln\left(\frac{V_t}{V_{t-1}}\right) = \ln(w_{t-1}^T y_t) + \ln\left(1 - \lambda \sum_{i=1}^{M} \left| w_{t,i} - w'_{t-1,i} \right| \right)$$

For small λ, $\ln(1 - x) \approx -x$, yielding $R_t \approx \ln(w_{t-1}^T y_t) - \lambda \cdot \text{turnover}_t$. This penalizes high turnover while maximizing the log-utility of returns, leading to policies that achieve high terminal wealth V_T with reduced trading frequency.

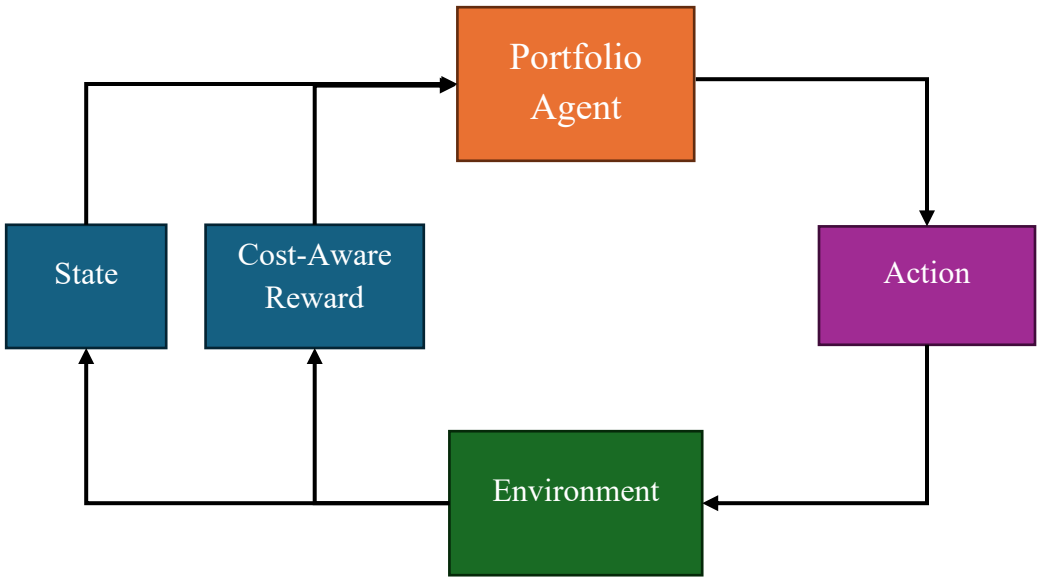

Figure 1. Proposed System Model

3.3 Algorithm Architecture: DDPG / PPO

Actor-critic methods handle the continuous action space by making use of either Deep Deterministic Policy Gradient (DDPG) or Proximal Policy Optimization (PPO), which is chosen based on empirical stability.

3.3.1 Actor Network

It encodes the policy $\mu(s_t|\theta^{\mu})$ by mapping states to weights w_t. The price tensor is first processed with the convolutional layers, followed by the dense layers that take the previous weights as input.

3.3.2 Critic Network

It calculates the state-action value $Q(s_t, a_t|\theta^{Q})$, which is the amount of expected future rewards (discounted) from the current state-action pair.

3.3.3 Cost-Aware Integration

High-turnover actions lead to lower R_t, which in turn lowers Q-values and directs the actor update (though policy gradient) towards low-turnover policies in similar states, as transaction costs will be computed in the environment's reward.

DDPG is applicable for off-policy learning combined with deterministic actions (along with exploration noise), while PPO provides on-policy stability through clipped objectives, thus these two are common choices in finance DRL (Liang et al., 2018).

Additional enhancements may include target networks, experience replay (for DDPG), or entropy regularization for exploration.

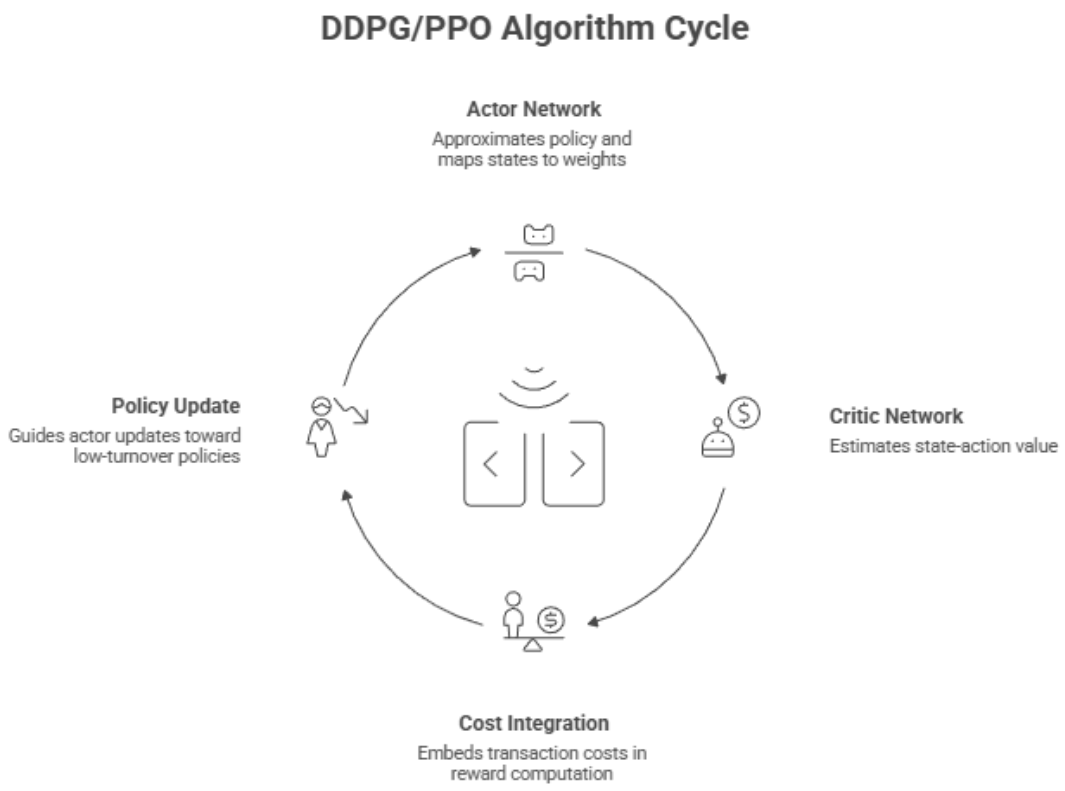

Figure 2. Algorithm Architecture

3.4 Data and Preprocessing

- **Data Source:** Daily OHLCV data retrieved via Yahoo Finance API (or equivalent libraries like yfinance).
- **Assets:** Either the 30 constituents of the Dow Jones Industrial Average (for blue-chip stability) or the top 10 cryptocurrencies by market cap (for higher volatility testing).
- **Period:** January 1, 2015, to January 1, 2023.
- **Normalization and Features:** Prices are normalized (e.g., relative changes or z-scores) per asset. Additional technical indicators (e.g., RSI, MACD) can augment the state for richer signals.
- **Splitting:**
    - Training (70%): 2015 - 2020 -- Policy learning on in-sample data.
    - Validation (15%): 2020 - 2021 -- Hyperparameter tuning (e.g., network size, λ, learning rates).
    - Testing (15%): 2021 - 2023 -- Out-of-sample evaluation to assess generalization.

Walk-forward validation may supplement to mimic real deployment.

3.5 Evaluation Metrics

Performance is benchmarked against classics like Buy-and-Hold (market index) and periodic Mean-Variance Optimization (rebalanced quarterly).

- **Accumulated Rate of Return (ARR):** $(V_T/V_0) - 1$.
- **Sharpe Ratio:** Risk-adjusted return

$$Sharpe = \frac{E\left[R_p - R_f\right]}{\sigma_p}\sqrt{252}$$

  (annualized, with risk-free rate ($R_f \approx 0$) for simplicity or using Treasury yields).

- **Maximum Drawdown (MDD):** Peak-to-trough decline,

$$MDD = \max_\tau\left(\frac{Peak_\tau - Trough_\tau}{Peak_\tau}\right)$$

- **Portfolio Turnover Rate:** Average daily/period turnover,

$$Turnover = \frac{1}{T}\sum_t\sum_i\left|w_{t,i} - w'_{t-1,i}\right|$$

  Critical for verifying cost-awareness.

Additional metrics: Calmar Ratio (ARR / MDD), Sortino Ratio (downside deviation).

3.6 Development Tools

In order to get the same results and train the Deep Reinforcement Learning models efficiently, the tests will be run in a controlled computational environment that is controlled. This part of the document describes the hardware and software tools used.

3.6.1 Hardware Specifications

- GPU Accelerator: NVIDIA RTX 4090 (24 GB VRAM) or equivalent. The use of GPU acceleration is a must for training actor-critic networks efficiently, hence the reduction of the episode training time from days (CPU-only) to hours.
- CPU: Intel Core i9-13900K processor (or AMD Ryzen 9 of similar power) is equipped with at least 16 cores for simultaneous data preprocessing and environment simulation.
- RAM: 64 GB DDR5 is required to accommodate large historical datasets and experience replay buffers without slow performance from excessive swapping.
- Storage: A 2 TB NVMe SSD for fast data loading and storing trained models/checkpoints.
- Alternative Cloud Setup: For the purpose of scalability, similar experiments can be conducted on cloud platforms like Google Colab Pro+, AWS EC2 (g5.xlarge instances), or Paperspace Gradient, providing GPU resources on-demand.

These specifications support consistent training of deep networks with convolutional and dense layers while simultaneously conducting thousands of trading episodes over multi-asset portfolios.

3.6.2 Software Specification

The system is relying on the use of Python-based open-source libraries that are already familiar in the quantitative finance and reinforcement learning communities.

Python: Version 3.10 or newer

Deep Reinforcement Learning Frameworks:

- Stable-Baselines3 (for PPO implementation) or CleanRL (for custom DDPG).
- Ray RLlib (optional, for distributed training and hyperparameter tuning)

Environment and Data Handling:

- FinRL library (Liu et al., 2021) as the base framework for portfolio management environments, providing pre-built MDP formulations, transaction cost modeling, and benchmarking tools.
- yfinance or pandas_datareader for downloading historical OHLCV data from Yahoo Finance.
- Pandas and NumPy for data preprocessing and feature engineering.

Neural Network Backend: PyTorch 2.3+ (preferred for dynamic computation graphs and GPU support in actor-critic architectures).

Visualization and Analysis:

- Matplotlib and Seaborn for drawing equity curves, drawdowns, and weight allocations.
- TensorBoard for the monitoring of training metrics (rewards, losses, turnover).

Additional Utilities: Jupyter Notebooks for interactive development, Git for version control, and Weights \& Biases (wandb) for experiment tracking and hyperparameter logging.

The entire codebase will be developed in a modular form to allow for reproducibility. The entire codebase that includes the environmental settings and the trained models will be uploaded to a public GitHub repository at the end of the project.

This setup ensures that the proposed cost-aware modifications can be rigorously tested against baselines while maintaining computational efficiency and adherence to best practices in reproducible RL research.

3.7 Expected Outcome

The Transaction Cost-Aware DRL agent is anticipated to:

- Demonstrate significantly lower turnover than a baseline DRL without explicit cost penalties, reducing real-world fees.
- Outperform Mean-Variance in net Sharpe Ratio (post-costs), leveraging non-linear pattern recognition in volatile or regime-shifting markets.

- Mitigate churning by learning stable allocations, enhancing practical deployability for automated trading systems while maintaining competitive returns and risk control.

CHAPTER FOUR

4. WORK AND BUDGET PLAN

4.1 Time Schedule

To achieve the overall objectives of this research work, it is important to divide the whole tasks in to more manageable, smaller tasks.

Table 4.1. Time plan of research work

No.	Tasks	Months																					
		Aug to Sept				Oct		Nov		Dec to Jan		Feb				Mar		Apr		May to June			
		Weeks																					
		1 2	3 4	1 2	3 4	1 2	3 4	1 2	3 4	1 2	3 4	1 2	3 4	1 2	3 4	1 2	3 4	1 2	3 4	1 2	3 4	1 2	3 4
1	Title articulation																						
2	Proposal writing																						
3	Proposal Defense																						
4	Literature review																						
5	System design																						
6	Simulation																						
7	Result and discussion																						
8	Thesis writing and editing																						
9	Submission document and final defense																						

4.2 Budget Breakdown

Table 4.2. Budget breakdown for research work

No.	Material Required	Unit	Quantity	Unit Price	Total Price (ETB)	Remark
Stationary Service						
1	A4 paper	Package	1	900	900	
	Personal Expense					
2	Per-diem for the research from Adama to Addis Ababa	Days	17	549	9,333	
3	Payment for data collector	Days	7	501	3,507	
Transportation Expense						
4	Transportation	Terms	11	260	2,860	
Secretarial service						
5	Printing	Package	6	200	1,200	
6	Final paper binding	Package	6	1200	7,200	
				Total	25,000	

Reference

Bai, Y., Gao, Y., Wan, R., Zhang, S., & Song, R. (2024). *A Review of Reinforcement Learning in Financial Applications*. http://arxiv.org/abs/2411.12746

Constantinides, G. M. (1986). Capital market equilibrium with transaction costs. *Journal of Political Economy*, *94(4)*, 842--862.

Coriat, B., & Benhamou, E. (2025). *HARLF: Hierarchical Reinforcement Learning and Lightweight LLM-Driven Sentiment Integration for Financial Portfolio Optimization*. http://arxiv.org/abs/2507.18560

Davis, M. H. A., & Norman, A. R. (1990). Portfolio selection with transaction costs. *Mathematics of Operations Research*, *15(4)*, 676--713.

Jiang, Y., Olmo, J., & Atwi, M. (2024). Deep reinforcement learning for portfolio selection. *Global Finance Journal*, *62*. https://doi.org/10.1016/j.gfj.2024.101016

Jiang, Z., Xu, D., & Liang, J. (2017). *A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem*. http://arxiv.org/abs/1706.10059

Liang, Z., Chen, H., Zhu, J., Jiang, K., & Li, Y. (2018). *Adversarial Deep Reinforcement Learning in Portfolio Management*. http://arxiv.org/abs/1808.09940

Liu, X.-Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., & Wang, C. D. (2022). *FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance*. http://arxiv.org/abs/2011.09607

Lwele, E., Emmanuel, S., & Sitali, S. G. (2025). *Risk-Aware Deep Reinforcement Learning for Dynamic Portfolio Optimization*. http://arxiv.org/abs/2511.11481

Markowitz, H. (1952). Portfolio Selection. In *The Journal of Finance* (Vol. 7, Issue 1).

Nawathe, S., Panguluri, R., Zhang, J., & Venkatesh, S. (2024). *Multimodal Deep Reinforcement Learning for Portfolio Optimization*. http://arxiv.org/abs/2412.17293

Ndikum, P., & Ndikum, S. (2024). *Advancing Investment Frontiers: Industry-grade Deep Reinforcement Learning for Portfolio Optimization*. http://arxiv.org/abs/2403.07916

Xue, P., & Ye, Y. (2025). *Attention-Enhanced Reinforcement Learning for Dynamic Portfolio Optimization*. http://arxiv.org/abs/2510.06466